

[SYSTEM] ALL DONE... 100% REQUIREMENTS MET

코디세이 b2-02 Git 실전 협업 시뮬레이션 최종 발표

한국 음식 128선 식사 추천 이상형 월드컵 (Python 게임 모듈)

Codysey B2 Team 2

👑 안재현 | 🛠 강동하 | 📋 김진우

완벽한 규칙이 완벽한 협업을 만든다. 일요일 15:00, 요구사항 100% 충족
조기 마감을 달성한 팀의 Git 협업 아키텍처와 트러블슈팅 실전 기록.

01. 협업의 기반: 3인 R&R 및 저장소 인프라 구축

👑 안재현: 총괄/엔진 (토너먼트 코어 및 게임 결합 리드)

🔧 강동하: DB/로더 (음식 128선 DB 및 로더 구축)

📋 김진우: 투표/문서 (다인원 투표기 및 최종 제출 인덱스 완결)

📁 Repository Root

- 📁 docs/ (협업 가이드/로그)
- 📁 src/ (파이썬 모듈)
- 📁 data/ (JSON DB)

Classic Branch Protection Rule: main

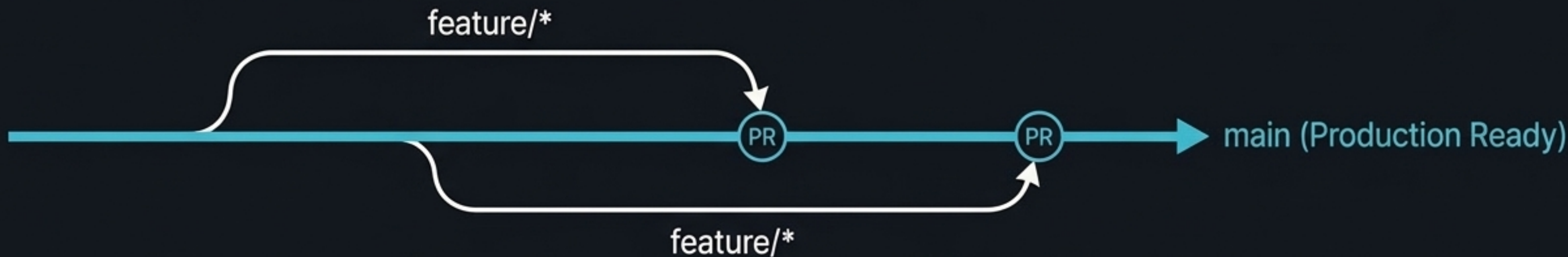
✅ 직접 push 차단 (PASS)

✅ Pull Request 생성 필수 (PASS)

✅ 최소 1인 이상 승인(Approve) 강제 (PASS)

출근 및 가용 일정을 분석하여 핵심 모듈별로 역할을 명확히 분담하고,
사고 방지를 위해 가장 엄격한 형태 🛡️의 main 브랜치 보호 규칙을 선제적으로 적용했습니다.

02. 브랜치 전략: 기민함과 안정성의 조화, GitHub Flow



“



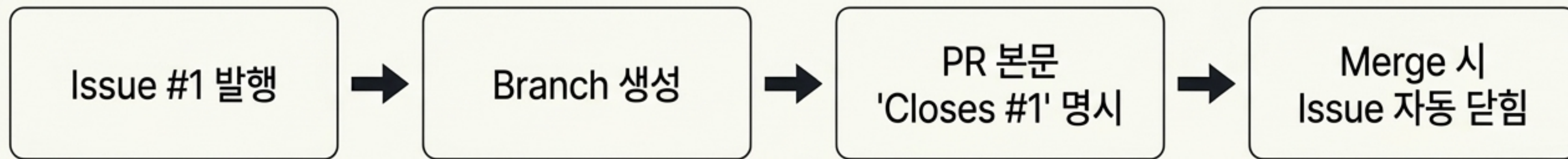
우리 팀이 GitHub Flow를 선택한 이유

1. 단순하고 명확한 흐름: 복잡한 분기 없이 기능 단위 브랜치와 main만 운용하여 협업 오버헤드 최소화.
2. 배포 상시성 보장: main은 언제나 배포 및 시연 가능한 깨끗한 상태 유지.
3. PR 중심 품질 관리: 모든 변경이 1인 이상 동료 리뷰를 거치도록 강제하여 결함 조기 차단.

”

단기 시뮬레이션 과제에 복잡한 Git Flow 대신,
가볍고 신속하며 상시 배포 가능한 GitHub Flow를 채택했습니다.

03. 작업의 투명화: 이슈 생애주기와 커밋 컨벤션



✓ 허용 (Type)

feat: (새로운 기능)
fix: (버그 수정)
docs: (문서 수정)
refactor: (구조 개선)
test: (테스트 코드)

⊘ 차단 (블랙리스트)

의미 없는 커밋 원천 금지:
update
fix (단독 사용)
temp
wip
final

코드 작성 전 Issue 발행을 의무화하여 추적성을 확보하고, 엄격한 커밋 규칙으로 히스토리 오염을 방지했습니다.

04. PR 기반 협업: 전원 달성 기여도 매트릭스

팀원	🔄 PR 생성/머지 (각 2개)	📝 코드 리뷰 작성 (각 2개)	🔄 피드백 반영 커밋
👑 안재현	🟢 완료 (PR #2, #12, #15)	🟢 완료	🟢 완료 (1회 이상)
🔧 강동하	🟢 완료 (PR #6, #9, #11)	🟢 완료	🟢 완료 (1회 이상)
📋 김진우	🟢 완료 (PR #4, #19)	🟢 완료	🟢 완료 (1회 이상)

무임승차 제로. 치밀한 타임라인 설계를 통해 팀원 3명 전원이 과제 최소 요구 조건인 'PR 2개, 리뷰 2개, 피드백 반영 1회'를 단 하나의 누락 없이 **100% 완벽히 달성**했습니다.

05. 리뷰의 품격: No LGTM, 실질적 성장 도구

```
24 + result = calculate_average(food_list)
25     return result
```

 강동하 (Reviewer) ...
여기 빈 리스트가 전달되면 ZeroDivisionError 발생 리스크가 있습니다.
리스트 길이를 사전에 체크하는 예외 처리 방어 로직 추가를 제안합니다.

 안재현 (Author)
좋은 지적 감사합니다. 방어 로직 추가했습니다.

 fix: Add empty list handling based on review

 **No
LGTM**

의미 없는 'LGTM(Looks Good To Me)' 단독 코멘트를 철저히 금지하고,
경계값 검증과 예외 상황을 짚어내는 라인 단위 인라인 리뷰 문화를 정착시켰습니다.

06. 충돌 극복 1: 인접 라인 동시 수정 (Keep Both)

```
<<<<<<< HEAD
{"id": 10, "name": "부대찌개", "desc": "1_햄과 소시지..."} (강동하: 테스트 오타자)
=====
{"id": 11, "name": "순두부찌개", "desc": "부드러운 식감..."} (안재현: 데이터 추가)
>>>>>>> feature/ahn-dish
```

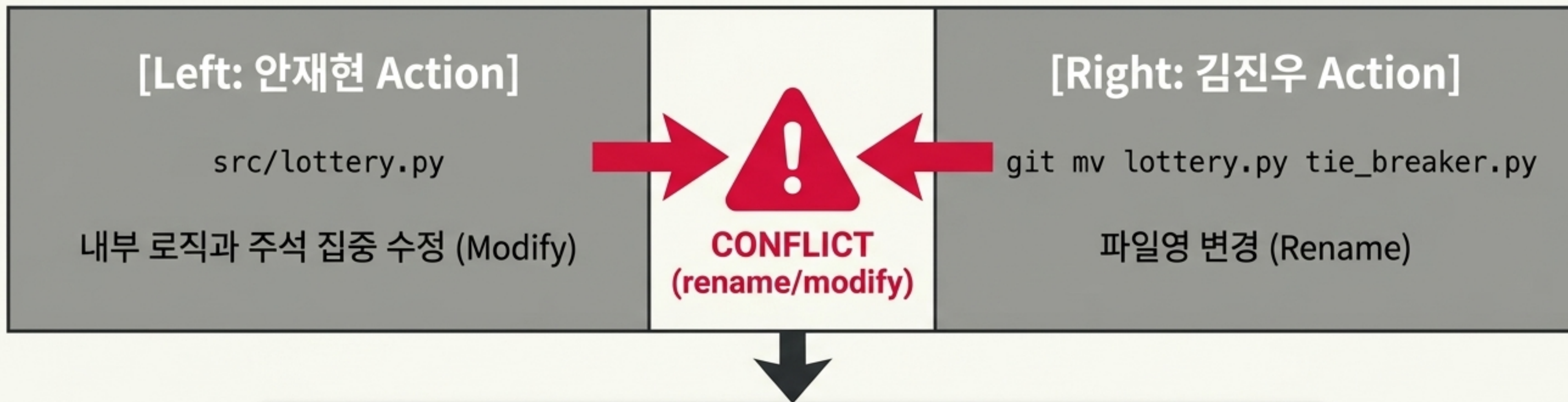


```
{"id": 10, "name": "부대찌개", "desc": "햄과 소시지..."}
{"id": 11, "name": "순두부찌개", "desc": "부드러운 식감..."}
```

해결 시퀀스:

1. 충돌 감지 (동일 라인 수정)
2. 오타자(1_) 정제
3. Keep Both 전략
(두 데이터 모두 살림)
4. 수동 병합 커밋 완료

07. 충돌 극복 2: 비자명(Non-Trivial) 충돌의 우아한 해결



우아한 3-Way 병합 해결 시퀀스

1. 김진우의 리네임(tie_breaker.py)을 프로젝트 표준으로 채택
2. 안재현의 수정 로직을 새 파일 내부로 이식
3. 기존 잔여 파일(lottery.py) git rm 정리
4. **완벽한 병합 커밋** 달성 (코드 유실 Zero)

08. Git 트러블슈팅: 4종 구조대 시나리오 실측

[1] git commit --amend

실행: 강동하 | 상황: 최근 커밋 오타 수정

결과: 원격 푸시 전 히스토리를 깔끔하게 교체하며 Tree 해시 보존 증명.

[2] git reset --soft HEAD~1

실행: 안재현 | 상황: 임시 파일 실수 커밋

결과: 커밋 단위만 언커밋하고 작업물(Staged)은 100% 안전하게 보존 복구.

[3] git revert

실행: 안재현 & 김진우 | 상황: 원격 푸시 완료된 커밋 에러

결과: 기존 히스토리 파괴 없이 역커밋(영수증 커밋)을 발행하여 안전하게 롤백.

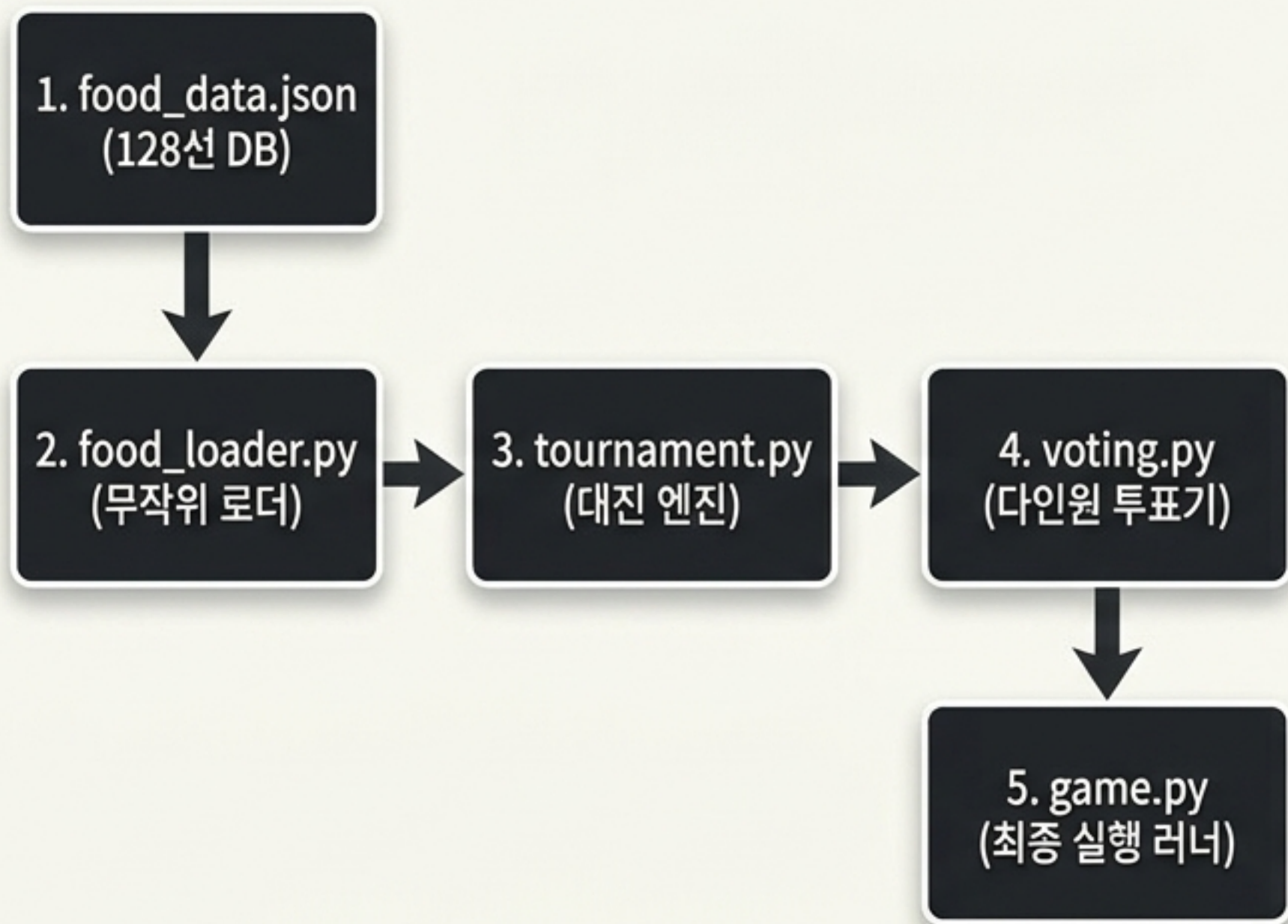
[4] git stash / pop

실행: 김진우 | 상황: 작업 중 급한 브랜치 전환 필요

결과: 미완성 WIP 코드를 임시 보관(스택)하고, 전환 작업 완료 후 정확히 복원 성공.

실무에서 마주할 수 있는 4가지 엣지 케이스를 상정하고, 전원이 참여하여 안전하게 복구하는 과정을 문서화했습니다.

09. 산출물: 한국 음식 128선 이상형 월드컵



```
$ python src/game.py --simulate
...
[SYSTEM] 토너먼트 시뮬레이션 완료. 최종 우승: 김치찌개
...
$ python -m unittest discover tests
.....
Ran 8 tests in 0.257s

OK!
```

Git 실습을 위한 단순 더미 코드가 아닌, 실제 128종의 메타데이터와 파이썬 모듈이 유기적으로 결합된 단일 실행 가능 CLI 게임을 완성했습니다.

10. 마침표: 무결점 Git Graph와 조기 완결 (ALL DONE)

```
* | Merge pull request #19
| \
| | * docs: Complete submission index
| | /
* | | Merge pull request #15
| | \ \
| | | * | fix: Resolve rename/modify conflict
| | | | * feat: Add advanced search functionality
| | | | /
* | | / | Merge branch 'feat/search'
| | /
* docs: Update README.md with final instructions
|
* feat: Finalize project structure and assets
|
* chore: Initial commit with base configuration
```

- SUBMISSION.md 대시보드
(모든 이슈/PR/충돌 링크 완비)
- 일요일 15:00 목표 시간 타격
- 요구사항 100% 충족
조기 완결 달성

ALL DONE

11. 결론 및 회고: Git은 약속이자 문화다

“
Git은 단순한 버전 관리 도구를 넘어,
코드를 매개로 팀의 신뢰를 구축하고
소프트웨어 엔지니어링 문화를 만들어가는
거대한 약속입니다.”

- 규칙 기반 협업(GitHub Flow)이 주는 통제력과 속도의 강력함 경험
- 'No LGTM' 원칙과 인라인 코드 리뷰를 통한 팀원 간 상호 기술적 성장
- 다양한 실전 충돌(Conflict)과 4종 트러블슈팅 복구를 통한 실무 장애 대응 자신감 확보